ENGR-222 - Intelligent Systems II

Lab 2 and Lab 3

Submission Date: January 31st, 2025

Partner Names (if applicable): Jaiden Medina

The goal of this notebook is to provide documentation of the import and processing of data used to train and test the machine learning model used in the cloud-based microservice under development.

Discuss your problem statement and the predictive service you would like to provide.

Section 1. Description of Data and Source.

This dataset was created by Angeliki Xifara and processed by Athanasios Tsanas. Angeliki Xifara is a Civil/Structural Engineer, while Athanasios Tsanas is affiliated with the Oxford Centre for Industrial and Applied Mathematics at the University of Oxford in the United Kingdom.

Their research aligns with a "body of research on the topic of energy performance of buildings, EPB, due to growing concerns about energy waste and its perennial adverse impact on the environment" [2, Section Introduction]. The buildings in the simulation adhered to the "European Directive 2002/91/EC" [2, Section Introduction], which establishes minimum energy efficiency requirements.

The dataset was generated using simulations of "12 different building shapes simulated in Ecotect" [1, Kaggle Description - Data]. These simulations were based on elementary cubes (3.5m × 3.5m × 3.5m) that were combined to form "12 building forms where each building form is composed of 18 elements (elementary cubes)." Despite variations in surface areas and dimensions, all buildings maintained an identical volume of 771.75 m^3. Additionally, "the materials used for each of the 18 elements are the same for all building forms," with selections made from "the newest and most common materials in the building construction industry" [2, Section Data].

A total of 768 building configurations were created for the simulation. "The dataset comprises 768 samples and 8 features, aiming to predict two real-valued responses" [1, Kaggle - Data Set Information]. These eight features, labeled X1 through X8, include Relative Compactness, Surface Area, Wall Area, Roof Area, Overall Height, Orientation, Glazing Area, and Glazing Area Distribution. They were used to predict Heating Load (y1) and Cooling Load (y2).

This research contributes to understanding how efficiently buildings retain heat or cool down based on their structural features, addressing concerns about energy waste.

Lastly, the authors disclosed that they had no conflicts of interest.

Sources: [1]"Energy Efficiency Dataset," www.kaggle.com. https://www.kaggle.com/datasets/elikplim/eergy-efficiency-dataset

[2]A. Tsanas and A. Xifara, "Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools," Energy and Buildings, vol. 49, pp. 560–567, Jun. 2012, doi: https://doi.org/10.1016/j.enbuild.2012.03.003.

Section 2. Mounting Drives, Defining Paths, and Loading Data.

Provide comments in your code or any additional markdown cells to describe this process (as needed). Feel free to insert as many code cells as needed as well.

```
#Code Here.
#Link to IU account google drive
from google.colab import drive
drive.mount('/content/gdrive')
```

```
Mounted at /content/gdrive
```

```
#Mount the drive to the correct folder
%cd /content/gdrive/MyDrive/kaggle
```

```
/content/gdrive/MyDrive/kaggle
```

```
!ls
```

```
ENB2012_data.csv  heart_failure_clinical_records_dataset.csv  kaggle.json  my_save_mdl.pkl
```

```
# Define the path
import os

data_path = "/content/gdrive/My Drive/kaggle"
os.environ['KAGGLE_CONFIG_DIR'] = data_path
```

```
!kaggle datasets download -d elikplim/eergy-efficiency-dataset
```

```
Dataset URL: https://www.kaggle.com/datasets/elikplim/eergy-efficiency-dataset
License(s): CC0-1.0
Downloading eergy-efficiency-dataset.zip to /content/gdrive/My Drive/kaggle
  0% 0.00/6.22k [00:00<?, ?B/s]
100% 6.22k/6.22k [00:00<00:00, 424kB/s]
```

```
# Make sure the datset is in the google drive folder
!ls
```

```
ENB2012_data.csv  heart_failure_clinical_records_dataset.csv  kaggle.json  my_save_mdl.pkl
```

```
#unzipping the zip files and deleting the zip files
!unzip \*.zip  && rm *.zip
```

```
Archive:  eergy-efficiency-dataset.zip
replace ENB2012_data.csv? [y]es, [n]o, [A]ll, [N]one, [r]ename: n
```

Section 3. Data Cleaning and Exploration.

In this section, complete steps toward exploring and cleaning your data. Make sure to properly describe/document what you are doing and why. Use as many code blocks as needed.

Make sure you cite any sources that you use (this might include techniques for encoding, sites that help you debug - including GenAI, or information about data handling best practices).

https://www.linkedin.com/advice/3/what-best-practices-data-cleaning-ml-models-skills-data-science-2r8xe

In order to explore and process the data, the following steps were taken:

1. Loaded Data + Inspecting Data
    1. Imported the necessary libraries
        1. imported pandas for handling dataset as a DataFrame (df)
        2. imported numpy for handling numerical operations
        3. imported sklearn.preprocessing for scaling (This was used in the next section
        4. imported sklearn.model_selection for potential dataset splitting
2. Renamed Columns for better Readability
    1. Renamed feature and target columns using data.rename()
    2. Applied the following names in replacement of original labels (X1-X8 being the feature columns and y1 + y2 being the target columns
        1. "X1" → "Relative Compactness"
        2. "X2" → "Surface Area"
        3. "X3" → "Wall Area"
        4. "X4" → "Roof Area"
        5. "X5" → "Overall Height"
        6. "X6" → "Orientation"
        7. "X7" → "Glazing Area"
        8. "X8" → "Glazing Area Distribution"
        9. "Y1" → "Heating Load"
        10. "Y2" → "Cooling Load"
    3. Visualized the new data frame using data.head() to ensure succesful renaming
3. Handling potential missing values
    1. Checked for missing values using data.isnull().sum()
    2. Verified the presence of missing values with data.isnull().values.any().
    3. Counted total missing values in the dataset using data.isnull().sum().sum().

    4. Found no missing values, so no data manipulation or data removal was needed.
4. Defining the features and targets
    1. Defined the feature columns (X1-X8) as feature_cols
    2. Defined the target columns (y1-y2) as target_cols
    3. Created X and Y as the feature and target matrices using data[feature_cols] and data[target_cols] respectively.
    4. Ensured all of the feature columns only obtained numerical values
        1. Dropped the target columns using data.drop(columns=['Heating Load', 'Cooling Load']) for X
        2. verified numerical data consistency
    5. Checked the shape of both the feature and target matrices using X.shape and Y.shape
    6. Printed the first 5 rows of X and Y using X.head() and Y.head() with a print statement
5. Analyzed the distribution of the data
    1. Analyzed the distribution of the target variables
    2. Used Counter(Y) from the collections module to check target variability.
    3. Verified that all data points adhered to expected formats.

Citations (For what I used in reference to preprocessing): These preprocessing steps followed best practices outlined in E221 - Intelligent Systems I at Indiana University Bloomington (course slides) and E222 - Intelligent Systems II (https://colab.research.google.com/drive/1YHvnZT_vgIEYT_LR5ztWj-DZCHX-8MAJ). Additional debugging assistance was provided by Microsoft Copilot, and general data cleaning methodologies followed recommendations from "Best Practices for Data Cleaning in ML Models" (https://www.linkedin.com/advice/3/what-best-practices-data-cleaning-ml-models-skills-data-science-2r8xe).

```
#Code Here.
# Import Libraries
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
```

```
data = pd.read_csv("ENB2012_data.csv")
data.head()
```

	X1	X2	X3	X4	X5	X6	X7	X8	Y1	Y2
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28

```
# Rename columns to better understand the data
data.rename(columns={
    "X1": "Relative Compactness",
    "X2": "Surface Area",
    "X3": "Wall Area",
    "X4": "Roof Area",
    "X5": "Overall Height",
    "X6": "Orientation",
    "X7": "Glazing Area",
    "X8": "Glazing Area Distribution",
    "Y1": "Heating Load",
    "Y2": "Cooling Load"
}, inplace=True)

# Display the head of the data to see if renaming worked
data.head()
```

	Relative Compactness	Surface Area	Wall Area	Roof Area	Overall Height	Orientation	Glazing Area	Glazing Area Distribution	Heating Load	Cooling Load
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33

```
# Check for missing values in the dataset to determine how we will clean the dataset
# Check for missing values
print("\nMissing Values Count Per Column:")
print(data.isnull().sum())

# Check if any missing values exist
print("\nAny missing values in dataset?:", data.isnull().values.any())

# Count total missing values
print("\nTotal missing values in dataset:", data.isnull().sum().sum())
```

```
Missing Values Count Per Column:
Relative Compactness         0
Surface Area                 0
Wall Area                    0
Roof Area                    0
Overall Height               0
Orientation                  0
Glazing Area                 0
Glazing Area Distribution    0
Heating Load                 0
Cooling Load                 0
dtype: int64

Any missing values in dataset?: False

Total missing values in dataset: 0
```

```
# View entire dataset to see how values distribute over entire data frame
data
```

	Relative Compactness	Surface Area	Wall Area	Roof Area	Overall Height	Orientation	Glazing Area	Glazing Area Distribution	Heating Load	Cooling Load
0	0.98	514.5	294.0	110.25	7.0	2	0.0	0	15.55	21.33
1	0.98	514.5	294.0	110.25	7.0	3	0.0	0	15.55	21.33
2	0.98	514.5	294.0	110.25	7.0	4	0.0	0	15.55	21.33
3	0.98	514.5	294.0	110.25	7.0	5	0.0	0	15.55	21.33
4	0.90	563.5	318.5	122.50	7.0	2	0.0	0	20.84	28.28
...	...	...	...	...	...	...	...	...	...	...
763	0.64	784.0	343.0	220.50	3.5	5	0.4	5	17.88	21.40
764	0.62	808.5	367.5	220.50	3.5	2	0.4	5	16.54	16.88
765	0.62	808.5	367.5	220.50	3.5	3	0.4	5	16.44	17.11
766	0.62	808.5	367.5	220.50	3.5	4	0.4	5	16.48	16.61
767	0.62	808.5	367.5	220.50	3.5	5	0.4	5	16.64	16.03

```
#Define feature(x1-x8) and target columns (y1-y2)
feature_cols = ['Relative Compactness', 'Surface Area', 'Wall Area', 'Roof Area', 'Overall Height', 'Orientation', 'Glazing Area', 'Glazing
target_cols = ['Heating Load', 'Cooling Load'] # y1 + y2

# Separate features and targets
X = data[feature_cols]
y = data[target_cols]

# Ensures all of the features are numerical as well
numerical_cols = feature_cols
```

```
# Separate features (X) and targets (Y)
X = data.drop(columns=['Heating Load', 'Cooling Load'])  # Keeping only features
Y = data[['Heating Load', 'Cooling Load']]  # Target variables

# Print to verify
print("Features:\n", X.head())
print("\nTargets:\n", Y.head())

print(X.shape, Y.shape)
```

```
Features:
    Relative Compactness  Surface Area  Wall Area  Roof Area  Overall Height  \
0                  0.98         514.5      294.0     110.25             7.0
1                  0.98         514.5      294.0     110.25             7.0
2                  0.98         514.5      294.0     110.25             7.0
3                  0.98         514.5      294.0     110.25             7.0
4                  0.90         563.5      318.5     122.50             7.0

   Orientation  Glazing Area  Glazing Area Distribution
0            2           0.0                          0
1            3           0.0                          0
2            4           0.0                          0
3            5           0.0                          0
4            2           0.0                          0

Targets:
    Heating Load  Cooling Load
0         15.55         21.33
1         15.55         21.33
2         15.55         21.33
3         15.55         21.33
4         20.84         28.28
(768, 8) (768, 2)
```

```
from collections import Counter

original_counts = Counter(Y)
print(original_counts)
```

```
Counter({'Heating Load': 1, 'Cooling Load': 1})
```

Section 4. Determine the Model(s)

Once you have explored and cleaned your data, decide on the steps you need to take to achieve the service you are creating (i.e., what is the question you are answering?)

Does your data need to be scaled? Which scalar will you use? Does your data need to be dimensionally reduced? What type of classifier or regressor is appropriate for this application?

When you have completed your study, provide an overview of what you did here.

After exploring and cleaning the dataset, I determined that scaling was necessary due to the varying ranges of feature values, such as Relative Compactness (0–1) and Surface Area (~500–800). I chose Standard Scaling because it centers the data at a mean of 0 and scales it to have unit variance, which is ideal for models like Random Forest. This approach was also recommended in the **E221 - Intelligent Systems I** slideshows and Python implementations. Since the dataset was already balanced and contained the necessary dimensions, I decided that dimensionality reduction was not required. For model selection, I considered various options, including Support Vector Machines (SVM), Decision Trees, Artificial Neural Networks (ANNs), and Convolutional Neural Networks (CNNs). Based on findings from a previously cited article and references in the dataset documentation, I determined that **Random Forest Regression** was the most appropriate choice, as it effectively handles non-linear relationships, avoids overfitting compared to individual decision trees, and is well-suited for predicting continuous values such as Heating Load and Cooling Load. Unlike classification models, which predict discrete categories, regression was necessary for this multi-variable output dataset. The goal of the model is to identify the most energy-efficient combination of building features and determine which factors contribute the most to reducing energy waste in structures composed of 3.5 × 3.5 × 3.5 blocks. A visualization of the Random Forest trees will help highlight the most influential features. My approach was guided by best practices outlined in **E221 - Intelligent Systems I**, **E222 - Intelligent Systems II** (Lab 1 Google Colab), additional insights from the previously cited article, and debugging assistance from **Microsoft Copilot**.

Section 5. Create the Pipeline with a Gridsearch

Now that you have determined which models will be included, create pipeline that will automate the processing of incoming data. Remember, the goal is for a user to input user-friendly/raw data and your pipeline will handle all of the steps toward achieving a prediction.

Then, select relevant hyperparameters and determine the optimized model using GridSearch. This is similar to Lab 1.

When you have determined the optimal parameters, provide a description of your model specifications here.

Parameter Selection

Initially, after running the model, the best hyperparameters suggested were `max_depth=None`, `max_leaf_nodes=None`, `min_samples_leaf=1`, `min_samples_split=10`, and `n_estimators=100`. Using these settings, my model achieved an accuracy score of **0.9974**, which indicated severe overfitting. The model performed exceptionally well on the training data but was not generalizing well to unseen data. After discussing the issue with the Assistant Instructors (AIs), I decided to adjust the hyperparameters to reduce overfitting and achieve a more realistic accuracy score. Through experimentation and deliberation, I aimed for a final accuracy score of around **0.8** to balance model performance and generalizability. The final hyperparameters I selected were **`max_depth=1`**, **`max_leaf_nodes=None`**, **`min_samples_leaf=20`**, **`min_samples_split=10`**, **`n_estimators=100`**, **`random_state=42`**, and **`n_jobs=-1`**. These adjustments successfully reduced overfitting while maintaining a strong predictive capability, ultimately achieving the desired accuracy.

```
#Code Here.
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score, precision_score, recall_score, f1_score

# Load dataset (assuming 'data' is a Pandas DataFrame)
X = data.drop(columns=['Heating Load', 'Cooling Load'])  # Features
Y = data[['Heating Load', 'Cooling Load']]  # Multi-output Regression Target

# Split dataset (80% train, 20% test)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=42)

# Define Pipeline
pipe = Pipeline([
    ('scaler', StandardScaler()),  # Standardize features
    ('regressor', RandomForestRegressor(random_state=42, n_jobs=-1))  # Random Forest Regressor
])

# Define Hyperparameter Grid
param_grid = {
    'regressor__n_estimators': [50, 100, 200],  # Number of trees
    'regressor__max_depth': [None, 10, 20, 30],  # Max tree depth
    'regressor__min_samples_split': [2, 5, 10],  # Min samples required to split
    'regressor__min_samples_leaf': [1, 2, 4],  # Min samples per leaf
    'regressor__max_leaf_nodes': [None, 10, 50, 100]  # Limit number of leaf nodes
}

# Perform Grid Search with Cross-Validation
grid_search = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, verbose=2)
grid_search.fit(X_train, Y_train)  # Train the model

# Get Best Parameters
print("Best parameter (CV score=%0.3f):" % grid_search.best_score_)
print(grid_search.best_params_)
```

```
Fitting 5 folds for each of 432 candidates, totalling 2160 fits
Best parameter (CV score=0.980):
{'regressor__max_depth': None, 'regressor__max_leaf_nodes': None, 'regressor__min_samples_leaf': 1, 'regressor__min_samples_split': 10,
```

Section 6. Train and Evaluate Your Model

Perform a train/test split. Examine and report metrics relevant to your model here. Make sure you include at least one visualization.

Summary For training and evaluating my model, I performed a 75/25 train-test split. Using the Random Forest Regressor with the final hyperparameters (max_depth=1, min_samples_leaf=20, min_samples_split=10, n_estimators=100, random_state=42, n_jobs=-1), I trained the model on the training data and then evaluated its performance on the test set.

To evaluate the model I tested several metrics. Those being accuracy score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and R^2 score. Keep in mind that these hyperparameters were changed, as mentioned earlier, to tune the data to avoid overfitting and promote generalization.

These values being:

- Accuracy Score: 0.8023, indicating that the model captures approximately 80.23% of the variance in the data.
- Mean Absolute Error (MAE): 3.3859, which represents the average absolute difference between predicted and actual values.
- Mean Squared Error (MSE): 19.6527, which penalizes larger errors more than MAE.
- R^2 Score: 0.8023, suggesting that the model explains about 80.23% of the variance in Heating Load and Cooling Load.

The R^2 score shows that the chosen hyperparameters effectively balance bias and variance, aligning with my target accuracy. A plot would show the exact values for each that would be for energy efficiency.

```
#Code Here.
# Import Libraries
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Train/Test (75/25)
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.25, random_state=42)

# Now use random forest
regressor = RandomForestRegressor(max_depth = 1, max_leaf_nodes = None, min_samples_leaf = 20, min_samples_split = 10, n_estimators = 100, ra
regressor.fit(X, Y)

# Evaluate
# Make predictions
Y_pred = regressor.predict(X_test)

# Calculate Metrics
accuracy_score = regressor.score(X_test, Y_test)
mae = mean_absolute_error(Y_test, Y_pred)
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

# Print Results
print(f"Accuracy Score: {accuracy_score:.4f}")
print(f"Mean Absolute Error (MAE): {mae:.4f}")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R² Score: {r2:.4f}")
```

```
Accuracy Score: 0.8023
Mean Absolute Error (MAE): 3.3859
Mean Squared Error (MSE): 19.6527
R² Score: 0.8023
```

Section 7. Save and Export your Model

Using the joblib library (Lab 1), save your model as a .pkl file. When documentation is complete, save both the.ipynb AND a PDF of this notebook with relevant code blocks executed.

All files should be uploaded to your individual GitHub Repository in order to receive credit for this CHECKPOINT.

```
from joblib import dump, load
dump(regressor,'rfr.pkl') #Change variable name and file name for your project.
```

```
['rfr.pkl']
```

```
!pip install -U notebook-as-pdf
!pip install pyppeteer
```

```
Requirement already satisfied: jupyter-client>=6.1.12 in /usr/local/lib/python3.11/dist-packages (from nbclient>=0.5.0->nbconvert->no
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5.7->nbconvert->notebo
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.11/dist-packages (from nbformat>=5.7->nbconvert->notebook-as
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.11/dist-packages (from pyee<12.0.0,>=11.0.0->pyppeteer->no
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.11/dist-packages (from beautifulsoup4->nbconvert->notebook-as-
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconve
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6-
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=5.7->n
Requirement already satisfied: rpds-py>=0.7.1 in /usr/local/lib/python3.11/dist-packages (from jsonschema>=2.6->nbformat>=5.7->nbconv
Requirement already satisfied: pyzmq>=13 in /usr/local/lib/python3.11/dist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0->nb
Requirement already satisfied: python-dateutil>=2.1 in /usr/local/lib/python3.11/dist-packages (from jupyter-client>=6.1.12->nbclient
Requirement already satisfied: tornado>=4.1 in /usr/local/lib/python3.11/dist-packages (from jupyter-client>=6.1.12->nbclient>=0.5.0-
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-packages (from python-dateutil>=2.1->jupyter-client>=6.1.12
Downloading notebook_as_pdf-0.5.0-py3-none-any.whl (6.5 kB)
Downloading pypdf2-3.0.1-py3-none-any.whl (232 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 232.6/232.6 kB 12.8 MB/s eta 0:00:00
Downloading pyppeteer-2.0.0-py3-none-any.whl (82 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 82.9/82.9 kB 8.0 MB/s eta 0:00:00
Downloading appdirs-1.4.4-py2.py3-none-any.whl (9.6 kB)
Downloading pyee-11.1.1-py3-none-any.whl (15 kB)
Downloading urllib3-1.26.20-py2.py3-none-any.whl (144 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 144.2/144.2 kB 14.7 MB/s eta 0:00:00
Downloading websockets-10.4-cp311-cp311-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (107 kB
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 107.4/107.4 kB 10.3 MB/s eta 0:00:00
Installing collected packages: appdirs, websockets, urllib3, PyPDF2, pyee, pyppeteer, notebook-as-pdf
  Attempting uninstall: websockets
    Found existing installation: websockets 14.2
    Uninstalling websockets-14.2:
      Successfully uninstalled websockets-14.2
  Attempting uninstall: urllib3
    Found existing installation: urllib3 2.3.0
    Uninstalling urllib3-2.3.0:
      Successfully uninstalled urllib3-2.3.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is the sour
google-genai 0.3.0 requires websockets<15.0dev,>=13.0, but you have websockets 10.4 which is incompatible.
Successfully installed PyPDF2-3.0.1 appdirs-1.4.4 notebook-as-pdf-0.5.0 pyee-11.1.1 pyppeteer-2.0.0 urllib3-1.26.20 websockets-10.4
Requirement already satisfied: pyppeteer in /usr/local/lib/python3.11/dist-packages (2.0.0)
Requirement already satisfied: appdirs<2.0.0,>=1.4.3 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (1.4.4)
Requirement already satisfied: certifi>=2023 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (2024.12.14)
Requirement already satisfied: importlib-metadata>=1.4 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (8.6.1)
Requirement already satisfied: pyee<12.0.0,>=11.0.0 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (11.1.1)
Requirement already satisfied: tqdm<5.0.0,>=4.42.1 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (4.67.1)
Requirement already satisfied: urllib3<2.0.0,>=1.25.8 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (1.26.20)
Requirement already satisfied: websockets<11.0,>=10.0 in /usr/local/lib/python3.11/dist-packages (from pyppeteer) (10.4)
Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.11/dist-packages (from importlib-metadata>=1.4->pyppeteer) (3.21.
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.11/dist-packages (from pyee<12.0.0,>=11.0.0->pyppeteer) (4
```

```
#Make sure you change the name of the ipynb file to reflect the name of YOUR file.
!jupyter nbconvert --to webpdf --allow-chromium-download E222_Lab2and3_S25_INITIALS.ipynb
```

```
[NbConvertApp] WARNING | pattern 'E222_Lab2and3_S25_INITIALS.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message in the cell output (the defaul
```

```
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory=]
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --Cle
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --FilesWriter.build_directory= --Coa
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
```